

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG



BÁO CÁO BÀI TẬP LỚN

HỆ THỐNG DỰ BÁO VÀ TRUY VẾT
NGUỒN Ô NHIỄM KHÔNG KHÍ THEO
THỜI GIAN THỰC

Học phần: **Lưu trữ và xử lý dữ liệu lớn**

Mã học phần: IT4931

Mã lớp: 168482

Giảng viên hướng dẫn: **TS. Trần Việt Trung**

Nhóm sinh viên thực hiện: **Nhóm 13**

STT	HỌ VÀ TÊN	MSSV
1	Lê Minh Hoàng	20230028
2	Nguyễn Ngọc Ánh	20235013
3	Nguyễn Minh Tùng	20235239
4	Nguyễn Ngọc Ánh	20235012
5	Phạm Hoàng Tiến	20230071

Hà Nội, 06/2026

Mục lục

1	Lời nói đầu	3
2	Bài toán và dữ liệu	4
2.1	Bài toán nghiệp vụ	4
2.2	Cơ sở dữ liệu khí quyển	4
2.3	Các tính chất Big Data	6
2.4	Cấu trúc dự án	8
3	Kiến trúc và thiết kế	9
3.1	Kiến trúc tổng thể	9
3.2	Pipeline dữ liệu	10
3.3	Data contract và lakehouse	13
3.4	Công nghệ sử dụng	16
4	Triển khai hệ thống	18
4.1	Ingestion và Kafka	18
4.2	Spark Streaming, Batch và Speed	19
4.3	Feature engineering	23
4.4	Trajectory và source attribution	25
4.5	Machine learning	26
4.6	Storage, API và dashboard	28
4.7	Airflow, Docker và Kubernetes	30
5	Vận hành và kiểm thử	34
5.1	Monitoring	34
5.2	Testing và data quality	34
5.3	Security và governance	35
5.4	Tối ưu hiệu năng	36
6	Kết quả triển khai	38
6.1	Kết quả chức năng	38
6.2	Kết quả thực nghiệm ở mức project	38
6.3	Điểm mạnh	39
6.4	Điểm cần cải thiện chính	39
7	Bài học kinh nghiệm	40
8	Phân công công việc	43
9	Kết luận	44
10	Tài liệu tham khảo	45

1 Lời nói đầu

Dự án **Atmospheric Intelligence System – AIS** là một hệ thống dữ liệu lớn được xây dựng để thu thập, xử lý, lưu trữ, phân tích, dự báo và trực quan hóa chất lượng không khí tại khu vực Hà Nội. Hệ thống tập trung vào chỉ số **PM2.5**, đồng thời tích hợp dữ liệu khí tượng, vệ tinh, tái phân tích khí quyển và feature trajectory sinh từ HYSPLIT để phục vụ phân tích nguyên nhân, bối cảnh khí tượng và khả năng vận chuyển ô nhiễm theo hướng gió.

AIS được thiết kế theo **kiến trúc Lambda đã hiệu chỉnh cho bài toán dữ liệu khí quyển**, triển khai trên nền lakehouse nhiều tầng. Batch layer xây dựng tầng Bronze/Silver/Gold, training dataset, lưu model artifact tại HDFS và quản lý production pointer bằng Iceberg. Speed layer được hiệu chỉnh thành hai đường chạy đồng thời: đường audit ghi Kafka event vào Iceberg Bronze, còn đường online kết hợp PM2.5/thời tiết mới nhất để cập nhật các đặc trưng trên Cassandra, và ghi lên API/UI.

Hệ thống sử dụng các thành phần cốt lõi của một dự án Big Data hiện đại, gồm Kafka cho message queue, Spark/Spark Structured Streaming cho xử lý batch và streaming, HDFS và Iceberg cho lakehouse storage, Cassandra/API/cache cho serving, Airflow cho orchestration, Docker Compose/Kubernetes cho triển khai, cùng monitoring và testing để theo dõi vận hành, chất lượng dữ liệu và khả năng phục hồi.

Sản phẩm cuối cùng là một pipeline hoàn chỉnh có khả năng:

1. Thu thập nhiều nguồn dữ liệu khí quyển.
2. Xử lý dữ liệu streaming và batch theo chuẩn lakehouse.
3. Tạo feature phục vụ dự báo PM2.5.
4. Huấn luyện, suy diễn cho mô hình dự báo.
5. Ghi dữ liệu phục vụ truy vấn nhanh vào Cassandra và cache visualization.
6. Cung cấp API và dashboard cho người dùng cuối.
7. Theo dõi trạng thái vận hành và phục hồi khi có lỗi.

Nhóm xin gửi lời cảm ơn chân thành tới **TS. Trần Việt Trung** đã hướng dẫn, góp ý và định hướng kỹ thuật trong quá trình thực hiện dự án. Những góp ý của thầy giúp nhóm hoàn thiện hơn về kiến trúc hệ thống, cách xử lý dữ liệu lớn, tư duy vận hành pipeline và cách trình bày một dự án Big Data hiện đại hoàn chỉnh.

2 Bài toán và dữ liệu

2.1 Bài toán nghiệp vụ

Bụi mịn PM2.5 là các hạt lơ lửng có đường kính khí động học không quá 2,5 micromet. Chúng có thể đi sâu vào phế nang, xâm nhập vào máu và gây hại cho hệ hô hấp, tim mạch. Năm 2021, Tổ chức Y tế Thế giới (WHO) khuyến nghị nồng độ PM2.5 trung bình 24 giờ không vượt quá $15 \mu\text{g m}^{-3}$ và trung bình năm không vượt quá $5 \mu\text{g m}^{-3}$ [11].

Tại Hà Nội, nồng độ PM2.5 thường vượt các ngưỡng trên, đặc biệt trong mùa gió mùa Đông Bắc từ tháng 10 đến tháng 3. Ô nhiễm hình thành từ sự kết hợp giữa phát thải giao thông, công nghiệp, đốt sinh khối với điều kiện khí tượng bất lợi và quá trình vận chuyển ô nhiễm từ xa. Vì vậy, dự báo PM2.5 ngắn hạn là bài toán phức tạp nhưng có giá trị thực tiễn cao.

Các nguồn dữ liệu mở như OpenAQ, ERA5, Sentinel-5P và MAIAC MODIS cung cấp thông tin bổ trợ về quan trắc mặt đất, khí tượng và khí quyển từ vệ tinh. Tuy nhiên, chúng khác nhau về định dạng, độ phân giải, chu kỳ cập nhật và yêu cầu kiểm soát chất lượng. Để hợp nhất dữ liệu đa nguồn, xử lý lịch sử và cập nhật gần thời gian thực, hệ thống cần sử dụng các công nghệ dữ liệu lớn phân tán.

Từ bối cảnh trên, dự án AIS đặt mục tiêu xây dựng hệ thống dữ liệu lớn có khả năng thu thập, xử lý, hợp nhất, phân tích, dự báo và giải thích diễn biến PM2.5 tại Hà Nội. Hệ thống tập trung giải quyết bốn thách thức kỹ thuật chính:

1. **Thu thập đa nguồn và mở rộng lưu trữ:** Thu thập ổn định, chuẩn hóa và lưu trữ dữ liệu khí quyển từ năm nguồn có schema, độ trễ và nhịp cập nhật khác nhau, đáp ứng cả xử lý dữ liệu lịch sử và gần thời gian thực.
2. **Đồng bộ theo thời gian và không gian:** Căn chỉnh dữ liệu khác múi giờ, độ phân giải, hệ tọa độ và cờ chất lượng thành không gian đặc trưng thống nhất, sẵn sàng cho phép nối dữ liệu và không làm lẫn thông tin tương lai.
3. **Dự báo theo nhiều khoảng thời gian:** Huấn luyện mô hình dự báo PM2.5 cho các mốc 6 giờ, 12 giờ và 24 giờ, đồng thời ngăn rò rỉ dữ liệu và bảo đảm mỗi đặc trưng chỉ được sử dụng khi đã thực sự khả dụng tại thời điểm huấn luyện hoặc suy diễn.
4. **Giải thích và truy nguyên bối cảnh ô nhiễm:** Bổ sung cho kết quả dự báo các bằng chứng khí tượng có thể diễn giải, gồm đường vận chuyển của khối khí đến Hà Nội và tín hiệu khí quyển quan sát được dọc theo đường đi đó.

2.2 Cơ sở dữ liệu khí quyển

2.2.1 Các nguồn dữ liệu khí quyển chính

AIS tích hợp các nguồn dữ liệu sau:

Nguồn	Vai trò trong hệ thống
OpenAQ	Nguồn quan trắc mặt đất chính cho PM2.5, dùng làm target và dữ liệu hiện trạng
WeatherAPI	Dữ liệu thời tiết bề mặt gần thời gian thực, dùng cho serving/inference khi ERA5 có độ trễ
ERA5	Dữ liệu tái phân tích khí quyển, cung cấp trường khí tượng ổn định cho batch feature và HYSPLIT
Sentinel-5P	Tín hiệu vệ tinh về NO2, CO, SO2, O3, aerosol index, hỗ trợ nhận diện bối cảnh ô nhiễm
MAIAC MODIS	AOD độ phân giải cao, bổ sung tín hiệu aerosol cột khí quyển

OpenAQ. OpenAQ tổng hợp dữ liệu quan trắc chất lượng không khí mở từ các mạng lưới của cơ quan quản lý và tổ chức nghiên cứu. Adapter ingestion của AIS truy vấn OpenAQ v3 REST API để lấy số đo PM2.5 theo giờ từ các trạm trong và xung quanh khu vực Hà Nội. Mỗi bản ghi gồm mã trạm, tọa độ, tên thông số, giá trị đo, đơn vị và thời điểm quan trắc. Hệ thống gán `event_id` ổn định (mã định danh duy nhất của sự kiện) từ `station_id` và `observed_at`, nhờ đó có thể gửi lại dữ liệu lên Kafka mà không tạo bản ghi trùng. Do số trạm trong nội thành chưa dày, AIS sử dụng thêm các trạm lân cận để xây dựng đặc trưng gradient không gian, phản ánh mức chênh lệch PM2.5 giữa các khu vực.

WeatherAPI. WeatherAPI cung cấp dữ liệu thời tiết bề mặt theo giờ cho danh sách địa điểm được cấu hình quanh Hà Nội, gồm nhiệt độ, điểm sương, độ ẩm tương đối, tốc độ và hướng gió, lượng mưa, tầm nhìn, chỉ số UV, độ che phủ mây và xác suất mưa. Nguồn này bổ sung cho ERA5 và đặc biệt hữu ích khi dự báo gần thời gian thực, do dữ liệu tái phân tích ERA5 thường được công bố chậm hơn vài ngày.

ERA5 tái phân tích khí quyển. ERA5 là bộ dữ liệu tái phân tích khí quyển toàn cầu do ECMWF xây dựng, cung cấp trạng thái khí quyển nhất quán về mặt vật lý từ năm 1940 đến gần hiện tại với độ phân giải ngang khoảng 31 km [6]. AIS thu thập hai nhóm sản phẩm. Nhóm bề mặt gồm độ cao lớp biên hành tinh, thành phần gió ở độ cao 10 m, nhiệt độ ở độ cao 2 m, áp suất bề mặt, tổng lượng mưa và tiêu tán lớp biên; các biến này được dùng trực tiếp làm đặc trưng khí tượng. Nhóm theo mực áp suất gồm gió, nhiệt độ và địa thế vị tại nhiều mực chuẩn từ 1000 hPa đến 50 hPa; dữ liệu được chuyển sang định dạng nhị phân NOAA ARL để làm đầu vào khí tượng cho HYSPLIT.

Sentinel-5P. Vệ tinh Sentinel-5P thuộc chương trình Copernicus mang theo thiết bị TROPOMI, cung cấp quan trắc toàn cầu hàng ngày về NO₂, CO, SO₂, O₃ và chỉ số aerosol [8]. Với mỗi granule (mảnh dữ liệu vệ tinh thu được trong một lần quét) đi qua khu vực Hà Nội, AIS tính trung bình có trọng số theo diện tích, độ lệch chuẩn và tỷ lệ điểm ảnh hợp lệ. Trước khi tính toán, dữ liệu được lọc theo ngưỡng đảm bảo chất lượng riêng của từng sản phẩm; ảnh hưởng của mây và các granule không phủ đúng khu vực được xử lý thông qua tỷ lệ điểm ảnh hợp lệ.

MAIAC MODIS (MCD19A2). Sản phẩm MAIAC MODIS cung cấp độ dày quang học aerosol (AOD, đại lượng biểu thị tổng lượng aerosol trong cột khí quyển) ở độ phân giải 1 km tại bước sóng 0,47 μm và 0,55 μm [9]. Trong điều kiện hòa trộn lớp biên phù hợp, AOD có tương quan với nồng độ PM_{2.5} gần mặt đất. AIS xử lý các granule HDF5, áp dụng bitmask chất lượng (các bit mã hóa trạng thái chất lượng phép đo) của MCD19A2 để chọn dữ liệu đáng tin cậy và tính AOD trung bình trên khu vực Hà Nội. Do dữ liệu gần như cập nhật hàng ngày nhưng nhạy với mây, pipeline sử dụng chính sách thời hạn hiệu lực (TTL) để nhận biết và xử lý dữ liệu thiếu hoặc đã cũ.

Các nguồn trên có định dạng, độ phân giải và độ trễ khác nhau, ví dụ JSON/API cho OpenAQ và WeatherAPI, NetCDF cho ERA5/Sentinel-5P và HDF5 cho MAIAC. Vì vậy, AIS cần các adapter ingestion, data contract và tầng Bronze/Silver/Gold để chuẩn hóa dữ liệu trước khi dùng cho ML và dashboard.

2.2.2 HYSPLIT và source attribution

HYSPLIT là mô hình được dùng để chạy **backward trajectory** từ khu vực Hà Nội, giúp phân tích đường đi của khối khí trong khoảng thời gian trước khi quan trắc PM_{2.5}. Từ kết quả mô phỏng trajectory, AIS tạo các feature như cluster đường đi, vùng centroid, hướng vận chuyển chính và tín hiệu vệ tinh dọc theo đường đi.

Source attribution được hiểu là phân tích hành lang vận chuyển tiềm năng. Hệ thống không khẳng định tuyệt đối một khu vực là nguồn phát thải gây ô nhiễm, vì để định lượng đóng góp phát thải cần thêm emission inventory hoặc mô hình hóa hóa học khí quyển như WRF-Chem/CMAQ. AIS sử dụng trajectory để tăng khả năng giải thích và cung cấp bối cảnh khí tượng cho dự báo PM_{2.5}.

2.3 Các tính chất Big Data

2.3.1 Volume

AIS lưu trữ đồng thời các lưới tái phân tích ERA5 dạng NetCDF ở nhiều mức áp suất, granule MAIAC dạng HDF5, sản phẩm Sentinel-5P dạng NetCDF và dữ liệu lịch sử nhiều năm từ OpenAQ, WeatherAPI. Tổng khối lượng dữ liệu vượt khả năng xử lý hiệu quả của

cơ sở dữ liệu quan hệ hoặc hệ thống lưu trữ đơn nút. Apache Iceberg trên HDFS cung cấp khả năng lưu trữ phân tán, quản lý snapshot và phân vùng dữ liệu, giúp truy vấn các khoảng lịch sử dài mà không phải quét toàn bộ bảng.

2.3.2 Velocity

OpenAQ và WeatherAPI liên tục cung cấp số đo mới theo giờ. Các adapter ingestion xuất bản sự kiện lên Kafka theo lịch cấu hình; Spark Structured Streaming tiêu thụ dữ liệu theo micro-batch và cập nhật bảng Bronze trong vài phút. Kafka tách tốc độ thu thập khỏi tốc độ xử lý của Spark, giúp hấp thụ các đợt dữ liệu tăng đột biến và cho phép phát lại dữ liệu khi cần backfill.

2.3.3 Variety

AIS xử lý sáu định dạng dữ liệu chính: JSON từ API, CSV từ các bản xuất lịch sử, GeoJSON cho lớp dữ liệu không gian, NetCDF4 cho ERA5 và Sentinel-5P, HDF5 cho MAIAC, cùng định dạng nhị phân NOAA ARL làm đầu vào khí tượng cho HYSPLIT. Mỗi định dạng cần parser hoặc bộ chuyển đổi riêng; các tầng ingestion và feature engineering che giấu sự khác biệt này, cung cấp giao diện dữ liệu dạng bảng thống nhất cho ML và trực quan hóa.

2.3.4 Veracity

AIS áp dụng kiểm soát chất lượng theo nhiều tầng và phù hợp với đặc điểm của từng nguồn: kiểm tra khoảng giá trị PM2.5 và độ phủ trạm cho OpenAQ; chuẩn hóa quy ước tích lũy lượng mưa của ERA5; áp dụng bitmask chất lượng cho MAIAC; lọc tỷ lệ điểm ảnh hợp lệ và ảnh hưởng của mây đối với Sentinel-5P; kiểm tra thời hạn hiệu lực của dữ liệu vệ tinh. Các bản ghi không đạt yêu cầu được gắn cờ chất lượng thay vì âm thầm loại bỏ, nhờ đó vẫn giữ được dấu vết phục vụ kiểm tra và truy vết.

2.3.5 Value

AIS tạo ra các giá trị vận hành chính sau:

- Cung cấp thông tin hiện trạng PM2.5 gần thời gian thực tại Hà Nội từ mạng lưới trạm quan trắc.
- Dự báo PM2.5 ngắn hạn tại các mốc 6 giờ, 12 giờ và 24 giờ.
- Phân tích mối liên hệ giữa PM2.5 với độ cao lớp biên, lượng mưa và chế độ gió.
- Phân tích nguồn gốc khối khí và hành lang vận chuyển tiềm năng bằng backward trajectory từ HYSPLIT.
- Cung cấp dashboard hỗ trợ theo dõi chất lượng không khí và ra quyết định trong y tế công cộng, quy hoạch đô thị.

2.4 Cấu trúc dự án

Cấu trúc dự án được tổ chức theo module chức năng:

Folder/File	Vai trò
ingest/	Adapter thu thập dữ liệu từ WeatherAPI, OpenAQ, Sentinel-5P, MAIAC, ERA5 và producer Kafka
spark_jobs/	Spark batch, Spark Structured Streaming, silver/gold/visualization/maintenance jobs
ml/	Train, promote, predict mô hình PM2.5
serving/	FastAPI PM2.5 API và visualization API
ui/	React/Vite dashboard
airflow/dags/	DAG điều phối toàn bộ pipeline
scripts/	Script bootstrap, check, submit job, sync Cassandra
deploy/k8s/	Kubernetes manifests, Spark-on-K8s templates, API/UI/ML jobs
config/	Runtime config, pipeline config, source config
docker-compose.yml	Môi trường local full-stack
Dockerfile	Build image runtime cho job/API
requirements.txt	Python dependencies
.env	Biến môi trường

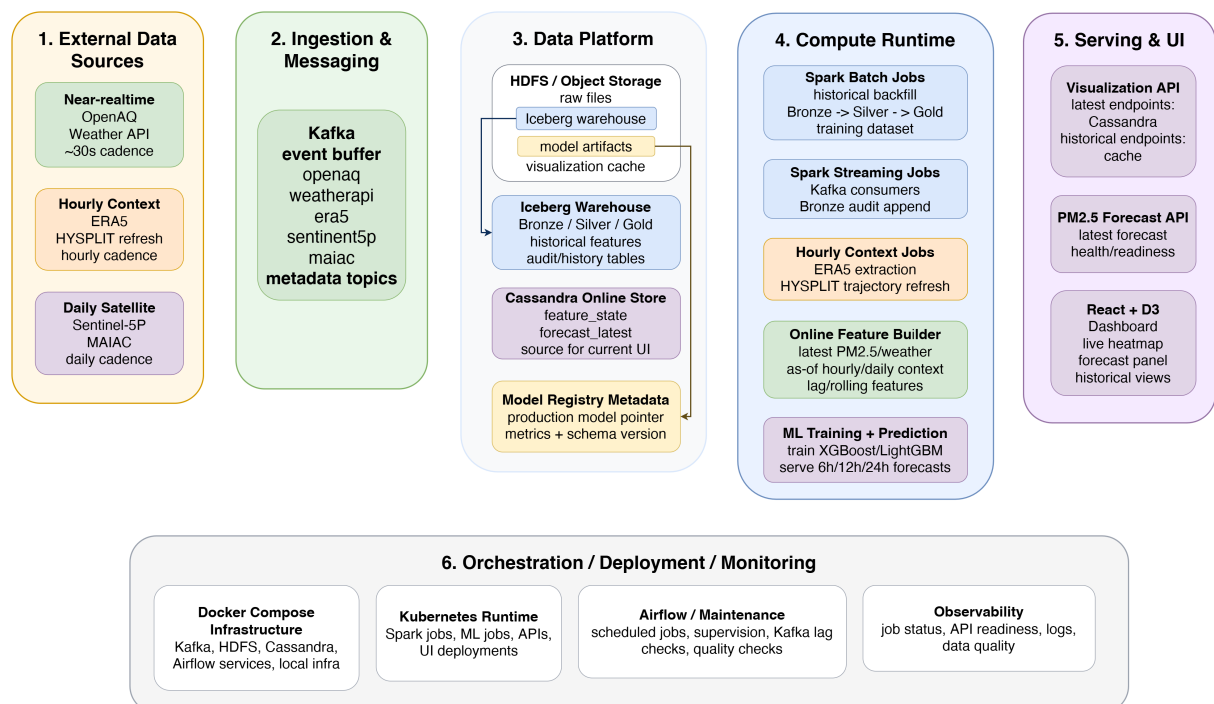
3 Kiến trúc và thiết kế

3.1 Kiến trúc tổng thể

3.1.1 Mô hình kiến trúc

AIS lấy cảm hứng theo kiến trúc Lambda, gồm batch layer, speed layer và serving layer, nhưng được hiệu chỉnh để phù hợp với đặc trưng đa tốc độ và độ trễ khác nhau của dữ liệu khí quyển. Batch layer xây dựng Bronze/Silver/Gold, training dataset và model. Speed layer không chỉ có một luồng xử lý tuần tự mà được tách thành hai đường gần thời gian thực chạy song song: Một luồng là streaming sink từ Kafka vào Iceberg Bronze để audit, lineage và replay; Luồng còn lại là online feature builder (thành phần tạo vector đặc trưng phục vụ dự báo từ dữ liệu mới nhất), kết hợp PM2.5/thời tiết mới nhất, lag/rolling và context as-of (bộ dữ liệu bối cảnh mới nhất đã khả dụng tại thời điểm dự báo, không sử dụng thông tin tương lai) từ vệ tinh, ERA5, HYSPLIT. Serving layer sử dụng Cassandra Feature State (kho lưu vector đặc trưng hiện hành theo địa điểm và thời điểm dự báo) và Cassandra Latest Forecast (kho lưu kết quả dự báo mới nhất theo địa điểm và thời hạn dự báo) để cung cấp dữ liệu độ trễ thấp cho API/UI. Như vậy, hệ thống sử dụng nguyên lý Lambda là kết hợp xử lý lịch sử đầy đủ với xử lý dữ liệu mới có độ trễ thấp, đồng thời bổ sung dual-path và context as-of để đáp ứng bài toán AIS.

3.1.2 Sơ đồ kiến trúc tổng thể



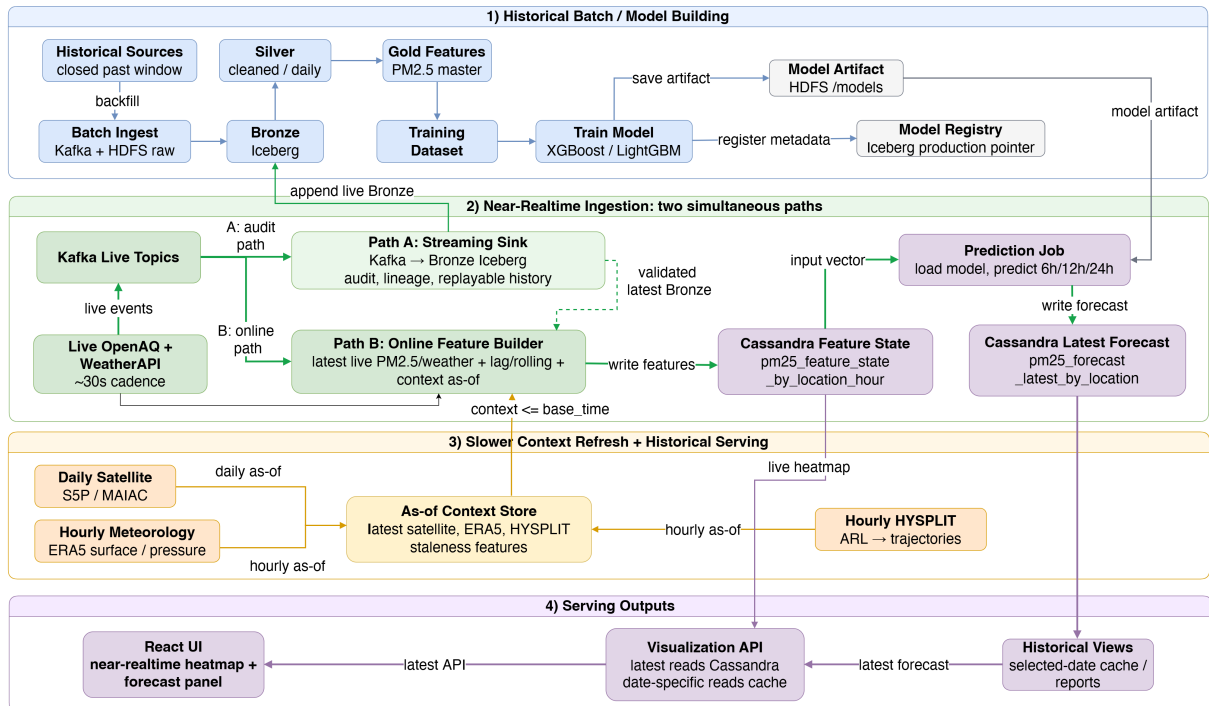
Hình 1: Kiến trúc tổng thể của hệ thống AIS

3.1.3 Vai trò các layer

Layer	Thành phần	Vai trò
External Data Sources	OpenAQ, WeatherAPI, ERA5, HYSPLIT, Sentinel-5P, MAIAC	Cung cấp dữ liệu theo cadence gần realtime, hourly và daily
Ingestion & Messaging	Kafka event buffer + HDFS raw	Nhận live event, metadata topic và raw file lớn
Data Platform	HDFS/Object Storage + Iceberg	Lưu raw, model artifact, visualization cache và Bronze/Silver/Gold lịch sử
Online Store	Cassandra Feature State + Latest Forecast	Lưu online feature vector và forecast mới nhất theo location
Model Registry	Iceberg production pointer	Quản lý metrics, schema version và con trỏ model production
Compute Runtime	Spark batch/streaming, context jobs, online feature builder	Backfill, Bronze audit append, refresh context as-of và tạo online features
ML	LightGBM + prediction job	Train model và dự báo PM2.5 6h/12h/24h
Serving & UI	Visualization API, PM2.5 Forecast API, React + D3.js	Đọc Cassandra cho latest, cache cho lịch sử và hiển thị dashboard
Orchestration	Airflow	Điều phối DAG, retry, backfill, dependency
Deployment	Docker/Kubernetes	Local dev và production-like runtime
Monitoring	Observability + Airflow maintenance	Theo dõi job, API readiness, log, Kafka lag và data quality

3.2 Pipeline dữ liệu

3.2.1 Luồng dữ liệu chính



Hình 2: Pipeline dữ liệu end-to-end của hệ thống AIS

3.2.2 Luồng batch

Luồng batch lấy historical sources trong closed past window, thực hiện batch ingest qua Kafka/HDFS raw rồi ghi Iceberg Bronze. Spark batch xử lý Bronze thành Silver, tạo Gold PM2.5 master features và training dataset. Job train LightGBM lưu model artifact tại HDFS /models, đồng thời đăng ký metadata và production pointer trong Iceberg Model Registry.

3.2.3 Luồng speed

Luồng gần thời gian thực nhận OpenAQ và WeatherAPI theo cadence rồi tách thành hai đường chạy đồng thời. Một luồng đưa Kafka live topics vào Bronze Iceberg để audit, lineage và replayable history. Luồng còn lại đưa live PM2.5/thời tiết trực tiếp vào Online Feature Builder, kết hợp lag/rolling, validated latest Bronze và context có thời điểm không vượt quá `base_time`. Feature vector được ghi vào Cassandra Feature State; Prediction Job tải model artifact production, dự báo 6h/12h/24h và ghi thẳng vào Cassandra Latest Forecast. Visualization API đọc Cassandra cho dữ liệu latest và đọc cache cho lịch sử; React UI không chạy Spark hoặc ML tại request time.

3.2.4 Pipeline theo từng bước

Bước	Thành phần thực hiện	Input	Xử lý	Output
1. Thu thập dữ liệu	<code>ingest/*.py</code>	API/file nguồn	Gọi API, download file, parse sơ bộ	Event JSON hoặc raw file
2. Publish queue	<code>ingest/kafka_utils.py</code>	Event chuẩn hóa	Serialize JSON, gán key <code>event_id</code> , retry/DLQ	Kafka topics
3. Raw storage	<code>ingest/era5_ingest.py</code> , Sentinel/MAIAC adapters	File lớn	Upload WebHDFS/HDFS	HDFS raw zone
4. Streaming Bronze	<code>spark_jobs/*_streaming.py</code>	Kafka topics	Parse schema, watermark, dedup, foreachBatch MERGE	Iceberg Bronze
5. Silver processing	<code>spark_jobs/*_silver.py</code>	Bronze/raw	Clean, QC, normalize, spatial/time align	Iceberg Silver
6. Trajectory tier	<code>hysplit_*</code> , <code>trajectory_*</code>	ERA5 pressure, satellite grid	ERA5→ARL, HYSPLIT, parse, cluster, sample path	Trajectory Silver/Features
7. Gold features	<code>hanoi_pm25_master_features_gold.py</code>	Silver datasets	Join, lag, rolling, calendar, satellite, trajectory	Master feature Gold
8. Training dataset	<code>hanoi_pm25_training_dataset_gold.py</code>	Master Gold	Point-in-time filter, split train/val/test, target shift	Training dataset Gold
9. ML training	<code>ml/train/promote</code>	Training dataset Gold	Train, evaluate, promote model	Model registry
10. Context refresh	Context jobs	ERA5, HYSPLIT, S5P/MAIAC	Cập nhật latest context và staleness theo as-of	As-of Context Store

Bước	Thành phần thực hiện	Input	Xử lý	Output
11. Online features	Online Feature Builder	Live PM2.5/ weather + Bronze + context as-of	Tạo lag/rolling và feature vector tại base_time	Cassandra Feature State
12. Online prediction	Prediction Job	Cassandra Feature State + production model artifact	Dự báo PM2.5 6h/12h/24h	Cassandra Latest Forecast
13. API/UI	Visualization API, Forecast API, React + D3	Cassandra latest + selected- date cache	Trả latest API, forecast, heatmap và historical view	Dashboard/ API response
14. Monitoring	Airflow + monitoring	Logs, metrics, state	Health, lag, freshness, DQ, alert	Ops dashboard

Pipeline đảm bảo tính idempotency ở các điểm ghi chính. Streaming Bronze dùng `foreachBatch + MERGE INTO` theo `event_id` hoặc natural key. Silver/Gold batch job ghi theo partition hoặc MERGE. Cassandra Feature State và Cassandra Latest Forecast dùng UPSERT theo location, base time và horizon. Nhờ đó cả luồng batch và hai near-realtime path đều có thể rerun/retry/backfill mà không tạo duplicate ngoài ý muốn.

3.3 Data contract và lakehouse

3.3.1 Event envelope và data contract

AIS sử dụng event envelope thống nhất cho dữ liệu Kafka:

```
{
  "event_id": "source-natural-key-hash",
  "source": "openaq|weather|sentinel5p|maiac|era5",
  "event_time": "2026-05-31T10:00:00Z",
  "ingest_time": "2026-05-31T10:02:15Z",
  "schema_version": "v1",
```

```

"available_at": "2026-05-31T10:02:15Z",
"payload": {},
"quality_flags": [],
"trace": {
  "request_id": "...",
  "producer": "...",
  "retry_count": 0
}
}

```

Mỗi bản ghi trong AIS mang theo một nhóm metadata thống nhất để hỗ trợ nối dữ liệu chính xác, kiểm soát chất lượng, truy vết và tái lập kết quả:

Nhóm metadata	Các trường chính	Mục đích
Định danh nguồn	source, event_id, location_id, sensor_id	Loại trùng và truy vết nguồn dữ liệu
Thời gian	observed_at, published_at, ingested_at, available_at	Nối dữ liệu đúng thời điểm
Chất lượng	unit, coverage_pct, valid_pixel_pct, qa_status, qa_flags	Chỉ chọn dữ liệu đạt yêu cầu chất lượng
Phiên bản	dataset_version, feature_version, schema_hash, model_version	Tái lập kết quả và phát hiện thay đổi
Truy vết	request_id, producer, job_run_id, input_snapshot_id	Debug và audit pipeline

3.3.2 Point-in-time correctness

Trong bài toán dự báo, hệ thống không được sử dụng dữ liệu tương lai. Một đặc trưng từ nguồn S chỉ được dùng để dự báo tại thời điểm T khi thỏa mãn:

```
S.available_at <= T
```

Quy tắc này đặc biệt quan trọng với ERA5 có độ trễ nhiều ngày, Sentinel-5P và MAIAC có thể trễ từ một đến hai ngày, cùng các đặc trưng trajectory được tạo từ ERA5 theo mức áp suất. Bảng Gold phục vụ huấn luyện chỉ lưu các đặc trưng thỏa mãn điều kiện trên tại từng thời điểm bắt đầu dự báo; bảng đặc trưng serving cũng kiểm tra điều kiện này trong job feature engineering trước khi suy diễn.

3.3.3 Bronze/Silver/Gold

Tầng	Ý nghĩa	Đặc điểm
Bronze	Dữ liệu gần thô sau ingestion	Giữ trace nguồn, event_id, ingest timestamp, payload chuẩn hóa tối thiểu
Silver	Dữ liệu sạch và chuẩn hóa	Schema validation, dedup, null handling, unit/time/coordinate normalization
Gold	Dữ liệu phục vụ ML/serving/visualization	Feature, forecast, dashboard products, model registry, cache manifest

3.3.4 HDFS raw zone

HDFS raw zone lưu các dữ liệu file lớn hoặc cần preserve bản gốc:

```
/raw/
|-- era5/
|   |-- surface/year=YYYY/month=MM/
|   `-- pressure/year=YYYY/month=MM/
|-- sentinel5p/
|   `-- product_date=YYYY-MM-DD/
|-- maiac/
|   `-- tile=.../date=YYYY-MM-DD/
`-- hysplit/
    `-- run_date=YYYY-MM-DD/
```

Raw zone giúp hệ thống tái xử lý dữ liệu khi thay đổi logic Silver/Gold, lưu bằng chứng nguồn dữ liệu, hỗ trợ backfill theo partition thời gian và tách dữ liệu file lớn khỏi Kafka.

3.3.5 Iceberg warehouse

Iceberg warehouse trên HDFS được tổ chức theo namespace:

```
warehouse/iceberg/
|-- ais.weather
|-- ais.air_quality
|-- ais.satellite
|-- ais.trajectory
|-- ais.features
|-- ais.models
```

```
|-- ais.predictions
|-- ais.visualization
```

3.3.6 Partitioning

Nhóm bảng	Partition chính	Lý do
Bronze event	event_date, source	Đọc theo ngày/nguồn, cleanup retention
OpenAQ silver	date, location_id	Truy vấn PM2.5 theo ngày/trạm
Weather silver	date, location_id	Join theo giờ và vị trí
ERA5 silver	year, month, variable	Dữ liệu file lớn theo tháng/biến
Gold features	feature_date, location_id	Train/predict theo thời gian/khu vực
Forecast	forecast_date, horizon	API query forecast nhanh
Visualization	product_date, product_type	Dashboard query theo sản phẩm

3.4 Công nghệ sử dụng

Công nghệ	Nhiệm vụ	File/module liên quan	Vai trò
Apache Spark / PySpark	Batch processing	spark_jobs/*_silver.py, spark_jobs/*_gold.py	Làm sạch, transform, join, aggregate
Spark Structured Streaming	Streaming processing	spark_jobs/*_streaming.py	Đọc Kafka và ghi Bronze idempotent
Apache Kafka	Message queue	ingest/kafka_utils.py, scripts/create_topics.sh	Buffer/replay event streaming
HDFS	Distributed storage	hadoop/, scripts/init_hdfs_layout.sh	Raw zone, checkpoint, warehouse
Apache Iceberg	Lakehouse table	ensure_iceberg_tables.py, Spark jobs	Bronze/silver/gold tables
Cassandra	Serving database	scripts/ensure_cassandra_online_schema.sh, pm25_serving_features_to_cassandra.py	Query nhanh cho API

Công nghệ	Nhiệm vụ	File/module liên quan	Vai trò
LightGBM	ML	ml/*.py	Dự báo PM2.5
FastAPI	API	serving/pm25_api/main.py, visualization API	REST endpoint
React/Vite	UI	ui/	Dashboard
Airflow	Orchestration	airflow/dags/*.py	Lập lịch, dependency, retry
Docker	Containerization	Dockerfile, docker-compose.yml	Local/runtime packaging
Kubernetes	Orchestration platform	deploy/k8s/	Production-like deployment

4 Triển khai hệ thống

4.1 Ingestion và Kafka

4.1.1 Tổng quan ingestion layer

Ingestion layer chịu trách nhiệm kết nối dữ liệu bên ngoài, chuẩn hóa event, quản lý trạng thái window, retry khi lỗi và gửi dữ liệu vào Kafka/HDFS.

File	Nguồn dữ liệu	Output chính
ingest/ingest_weather.py	WeatherAPI/local weather payload	Kafka weather
ingest/openaq_ingest.py	OpenAQ API	Kafka openaq
ingest/sentinel5p_ingest.py	Copernicus/CDSE OData + NetCDF product	Kafka sentinel5p-summary, HDFS raw
ingest/maiac_ingest.py	NASA CMR/MAIAC HDF5 granule	Kafka maiac-summary, HDFS raw
ingest/era5_ingest.py	CDS ERA5	HDFS raw + Kafka era5-files
ingest/kafka_utils.py	Kafka helper	Producer config, schema envelope, retry, DLQ
ingest/window_utils.py	Incremental state	Quản lý cửa sổ ingest theo nguồn

Mỗi ingestion adapter có cùng cấu trúc xử lý:

1. Đọc config nguồn dữ liệu, bbox, time window và credentials từ environment.
2. Gọi API hoặc download file với timeout/retry.
3. Chuẩn hóa timestamp về UTC.
4. Gán `event_id` ổn định từ natural key.
5. Gán `schema_version`, `ingest_time`, `available_at`, `quality_flags`.
6. Publish Kafka hoặc upload HDFS raw.
7. Ghi runtime state để incremental run tiếp tục từ mốc cuối.

4.1.2 Kafka topics

Topic	Producer	Consumer	Message
weather	ingest_weather.py	weather_streaming.py	Weather hourly event
openaq	openaq_ingest.py	openaq_streaming.py	PM2.5 measurement
sentinel5p-summary	sentinel5p_ingest.py	sentinel5p_summary_streaming.py	Sentinel product metadata
maiac-summary	maiac_ingest.py	maiac_summary_streaming.py	MAIAC granule metadata
era5-files	era5_ingest.py	era5_files_streaming.py	ERA5 file metadata

4.1.3 Kafka deployment scale

Thiết kế production đúng của hệ thống sử dụng **3 Kafka brokers** với replication factor 2–3, retention tối thiểu 7 ngày và partition count đủ để Spark consumer song song. Tuy nhiên, trong môi trường triển khai hiện tại của nhóm, hệ thống chạy cấu hình rút gọn **1 Kafka broker** để giảm RAM/CPU và phù hợp với giới hạn phần cứng máy cá nhân.

4.1.4 Message schema và offset handling

Message Kafka được validate theo schema version. Spark streaming định nghĩa schema rõ ràng bằng StructType trước khi parse JSON. Event không hợp lệ được ghi vào DLQ kèm raw payload và lý do lỗi. Offset Kafka được quản lý bằng Spark checkpoint trên HDFS, mỗi streaming job có checkpoint riêng dưới /checkpoints/spark_streaming/<job_name>.

4.2 Spark Streaming, Batch và Speed

4.2.1 Các streaming job chính

File	Topic input	Output	Chức năng
weather_streaming.py	weather	ais.weather.weather_bronze	Parse weather event, watermark, dedup
openaq_streaming.py	openaq	ais.air_quality.openaq_bronze	Parse PM2.5 event, validate station/time/value
sentinel5p_summary_streaming.py	sentinel5p-summary	ais.satellite.sentinel5p_summary_bronze	Parse product metadata, validate coverage/QA

File	Topic input	Output	Chức năng
maiac_summary_streaming.py	maiac-summary	ais.satellite.maiac_summary_bronze	Parse granule metadata, validate AOD QA
era5_files_streaming.py	era5-files	ais.weather.era5_files_bronze	Register ERA5 files by URI/checksum

4.2.2 Streaming read từ Kafka

Mỗi job đọc Kafka theo cấu hình:

- `subscribe`: topic tương ứng.
- `startingOffsets`: `latest` cho realtime, `earliest` cho backfill có kiểm soát.
- `failOnDataLoss`: cấu hình an toàn theo môi trường.
- `maxOffsetsPerTrigger`: giới hạn tốc độ đọc để tránh overload.
- `kafka.bootstrap.servers`: lấy từ config/environment.

4.2.3 Schema parsing và validation

Mỗi message Kafka ban đầu là một chuỗi JSON. Spark Structured Streaming ánh xạ chuỗi này vào schema được khai báo trước để xác định rõ tên trường, kiểu dữ liệu và cấu trúc lồng nhau. Cách làm này giúp phát hiện sớm các lỗi như thiếu trường bắt buộc, sai kiểu thời gian, giá trị không đúng định dạng hoặc phiên bản schema không được hỗ trợ.

Sau khi parse, job tách các trường metadata chung gồm `event_id`, `source`, `event_time`, `ingest_time`, `schema_version`, `available_at`, đồng thời đọc phần dữ liệu nghiệp vụ trong `payload` và các cờ chất lượng. Bản ghi hợp lệ được chuyển sang các bước watermark, loại trùng và ghi Bronze. Bản ghi không hợp lệ không làm dừng toàn bộ luồng mà được đưa vào DLQ kèm dữ liệu gốc và lý do lỗi để có thể kiểm tra, sửa và phát lại sau.

4.2.4 Watermark, deduplication và late data

Streaming job sử dụng watermark theo `event_time`, ví dụ:

```
withWatermark("event_time", "24 hours")
```

Deduplication được thực hiện theo `event_id` hoặc natural key tương ứng với từng nguồn. Watermark giúp xử lý dữ liệu đến trễ trong ngưỡng cho phép, giới hạn state store và đưa event quá trễ vào late-event table/DLQ thay vì làm hỏng toàn bộ stream.

4.2.5 Trigger interval, watermark và late-event policy theo stream

Mỗi nguồn dữ liệu có nhịp cập nhật và độ trễ khác nhau, vì vậy AIS không dùng một cấu hình streaming duy nhất cho tất cả các topic.

Stream	Topic	Trigger	Watermark	Late-event policy
OpenAQ PM2.5	openaq	1–5 phút	24 giờ	Trong watermark: MERGE vào Bronze/Silver; quá watermark: late table/DLQ
WeatherAPI	weather	1–5 phút	12 giờ	Cập nhật weather hourly; bản ghi quá muộn được gắn cờ stale
Sentinel-5P summary	sentinel5p -summary	Theo batch event	3 ngày	Chấp nhận late vì dữ liệu vệ tinh có độ trễ tự nhiên; dùng <code>available_at</code> khi join feature
MAIAC summary	maiac-sum mary	Theo batch event	3 ngày	Chấp nhận late, gắn freshness/valid pixel flags
ERA5 file metadata	era5-files	Theo batch ingest	7 ngày	Register file metadata; không dùng cho realtime nếu <code>available_at > base_time</code>

Với các stream realtime như OpenAQ và WeatherAPI, watermark ưu tiên độ mới và khả năng cập nhật nhanh. Với Sentinel-5P, MAIAC và ERA5, watermark dài hơn vì bản chất nguồn có độ trễ công bố. Các bản ghi đến muộn không bị xóa âm thầm mà được gắn `late_event`, `stale_feature` hoặc ghi vào DLQ/late table để phục vụ debug và backfill.

4.2.6 Các batch job chính

Spark batch được dùng cho xử lý file raw lớn, làm sạch dữ liệu Bronze thành Silver, tạo Gold feature, backfill dữ liệu lịch sử, recompute khi thay đổi logic và maintenance Iceberg.

File	Vai trò	Input	Output
ensure_iceberg_tables.py	Tạo namespace/table Iceberg	Config schema	Iceberg metadata
hanoi_openqa_silver.py	Chuẩn hóa PM2.5 OpenAQ	Bronze OpenAQ	Silver OpenAQ

File	Vai trò	Input	Output
hanoi_weather_surface_proxy_silver.py	Chuẩn hóa weather	Bronze weather	Silver weather
era5_surface_hanoi_silver.py	Parse ERA5 surface	HDFS raw ERA5	Silver ERA5
sentinel5p_hanoi_silver.py	Chuẩn hóa Sentinel-5P	Bronze/raw Sentinel	Silver Sentinel
maiac_hanoi_silver.py	Chuẩn hóa MAIAC/AOD	Bronze/raw MAIAC	Silver MAIAC
openaq_spatial_gradient_silver.py	Tính gradient không gian PM2.5	Silver OpenAQ	Silver spatial gradient
hanoi_pm25_master_features_gold.py	Tạo master features	Silver datasets	Gold master features
hanoi_pm25_training_dataset_gold.py	Tạo training dataset	Gold master	Gold training dataset
visualization*_gold.py	Tạo sản phẩm trực quan	Silver/gold	Visualization gold/cache
iceberg_maintenance.py	Optimize/expire snapshots	Iceberg tables	Optimized tables

Các batch job silver/gold sử dụng logic idempotent: recompute theo partition thời gian, MERGE INTO theo natural key, dynamic partition overwrite cho bảng aggregate và ghi audit metadata gồm `job_run_id`, `processing_time`, `input_snapshot_id`, `output_snapshot_id`. Điều này giúp rerun job không tạo duplicate và hỗ trợ lineage.

4.2.7 Các speed job chính

Near-realtime layer tách dữ liệu mới thành audit path và online path. Audit path duy trì Bronze replayable; online path kết hợp live event với context as-of để cập nhật state và forecast phục vụ API:

File	Vai trò	Input	Output
hanoi_pm25_serving_features_gold.py	Online Feature Builder	Live PM2.5/weather + latest Bronze + context as-of	Cassandra Feature State

File	Vai trò	Input	Output
ml/predict_hanoi_pm25.py	Chạy online prediction	Cassandra Feature State + production model artifact	Cassandra Latest Forecast
pm25_serving_features_to_cassandra.py	Đồng bộ online state/read model	Latest features theo location/hour	Cassandra Feature State

4.3 Feature engineering

4.3.1 Cleaning và validation

Nguồn	Kiểm tra chất lượng
OpenAQ	PM2.5 ≥ 0 , ngưỡng trên hợp lý, unit normalization, station/location validation, duplicate measurement
WeatherAPI	Timestamp hourly alignment, null interpolation giới hạn, wind direction/speed range, precipitation non-negative
ERA5	Kiểm tra file checksum, variable availability, spatial interpolation, precipitation accumulation convention
Sentinel-5P	QA threshold, valid pixel percentage, bbox intersection, cloud/coverage filtering
MAIAC	QA bitmask, valid AOD range, cloud-free pixel, tile/date validation
HYSPLIT output	Run completeness, trajectory age sequence, lat/lon range, missing waypoint handling

4.3.2 Join logic

Các join chính trong hệ thống:

Join	Input	Key	Output
PM2.5 + Weather	OpenAQ + weather hourly	event_hour, location_id	Weather-enriched PM2.5
PM2.5 + ERA5	OpenAQ + ERA5 hourly	event_hour, nearest grid/Hanoi center	Meteorological features
PM2.5 + Satellite	Hourly PM2.5 + daily satellite	event_date, point-in-time available_at	Daily satellite features

Join	Input	Key	Output
PM2.5 + Transport features	Hourly PM2.5 + features sinh từ HYSPLIT output	<code>event_hour</code> , <code>location_id</code>	Transport-enriched features
Serving features + Model registry	Serving feature table + production model	<code>feature_version</code> , <code>schema_hash</code> , <code>horizon</code>	Forecast output

Broadcast join được dùng cho dimension nhỏ như station metadata, grid metadata, location config. Join dữ liệu lớn được repartition theo time/location và tận dụng partition pruning trên Iceberg.

4.3.3 Master feature table

`hanoi_pm25_master_hourly_gold` là feature store trung tâm của AIS. Bảng này join tất cả nguồn Silver tại grain hourly cho Hà Nội.

Nhóm feature	Ví dụ
PM2.5 observed	median, mean, station_count, coverage_pct, valid_station_count
Lag features	pm25_lag_1h, pm25_lag_3h, pm25_lag_6h, pm25_lag_12h, pm25_lag_24h
Rolling statistics	rolling mean/std/max 3h/6h/24h
Weather	temperature, humidity, pressure, wind speed, wind direction, precipitation, visibility
ERA5	pbl_height, wind_u_10m, wind_v_10m, surface pressure, total precipitation
Sentinel-5P	NO2, CO, SO2, O3, aerosol index, valid_pct, freshness
MAIAC	AOD_047, AOD_055, valid_pct, freshness
Spatial gradient	north/south/east/west gradient, gradient_magnitude
Transport/HYSPLIT output	cluster_id, centroid, path NO2/AER mean, residence signal
Calendar	hour, day_of_week, month, season, weekend, rush hour, cyclical encoding

`hanoi_pm25_training_dataset_gold` và `hanoi_pm25_serving_features_gold` dùng cùng feature set và `schema_hash`. Training dataset có thêm target labels; serving feature table không chứa target tương lai và chỉ sử dụng feature thỏa mãn `available_at <= prediction_origin`. Nếu `schema_hash` không khớp model registry, inference bị chặn và sinh alert.

4.3.4 UDF tùy biến cho logic nghiệp vụ

Bên cạnh các hàm built-in của Spark SQL, AIS sử dụng UDF cho một số logic nghiệp vụ khó biểu diễn gọn bằng biểu thức SQL chuẩn:

UDF/logic	Mục đích
Chuẩn hóa hướng gió	Chuyển wind direction dạng độ sang nhóm hướng Bắc/Đông/Nam/Tây hoặc sector 8 hướng để phục vụ phân tích vận chuyển ô nhiễm
QA bitmask cho MAIAC	Giải mã bitmask chất lượng pixel AOD, chỉ giữ pixel đạt chất lượng đủ tốt
Phân loại vùng trajectory	Gán waypoint/endpoint vào nhóm vùng địa lý tương đối so với Hà Nội để phục vụ source attribution
Sinh quality flags	Gắn cờ <code>missing_value</code> , <code>out_of_range</code> , <code>low_coverage</code> , <code>late_event</code> , <code>stale_feature</code> theo quy tắc từng nguồn

UDF chỉ được dùng cho logic đặc thù khó thay bằng hàm Spark có sẵn. Với các thao tác phổ biến như lọc null, chuẩn hóa timestamp, tính lag/rolling, join và aggregate, hệ thống ưu tiên Spark SQL/DataFrame built-in để Catalyst Optimizer có thể tối ưu kế hoạch thực thi tốt hơn.

4.4 Trajectory và source attribution

4.4.1 Motivation

PM2.5 tại Hà Nội không chỉ phụ thuộc vào phát thải cục bộ và điều kiện khí tượng tại chỗ. Quá trình vận chuyển xa của aerosol và các khí tiền chất từ hoạt động đốt sinh khối ở Đông Nam Á, khu vực công nghiệp phía nam Trung Quốc hoặc các đợt bụi từ sa mạc Gobi có thể làm nồng độ PM2.5 tăng đáng kể. Ảnh hưởng này rõ hơn trong mùa khô, khi gió mùa Đông Bắc tạo điều kiện đưa các khối khí từ phía bắc về Hà Nội.

Phân tích backward trajectory bằng HYSPLIT cung cấp cơ sở vật lý để xác định đường đi của khối khí trước khi đến Hà Nội. Từ đó, hệ thống có thể bổ sung tín hiệu vận chuyển ô nhiễm vào dữ liệu dự báo, đồng thời hỗ trợ giải thích liệu một đợt PM2.5 cao có liên quan đến nguồn phát thải tại chỗ hay các hành lang vận chuyển từ xa.

4.4.2 HYSPLIT trajectory

HYSPLIT yêu cầu trường khí tượng ở định dạng nhị phân NOAA ARL. Trước khi chạy mô hình, job `era5_pressure_levels_to_arl.py` đọc dữ liệu ERA5 NetCDF theo nhiều mực áp suất, lấy các thành phần gió, nhiệt độ và địa thế vị, sau đó chuyển đổi sang

ARL. File đầu ra được lưu trên HDFS và metadata được ghi vào Iceberg để hỗ trợ truy vết và chạy lại.

Các tham số vận hành như điểm tiếp nhận, thời gian truy ngược, độ cao khởi phát và lịch chạy được quản lý tập trung trong cấu hình pipeline. Cách tổ chức này cho phép điều chỉnh phạm vi phân tích theo từng kịch bản mà không phải thay đổi mã nguồn của job.

Job `hysplit_trajectory_run.py` chạy HYSPLIT ở chế độ backward từ khu vực Hà Nội để xác định đường đi của khối khí trước khi đến điểm tiếp nhận. Mỗi lần chạy tạo một file trajectory ghi lại vị trí và các điều kiện khí tượng liên quan tại từng bước thời gian trong quá khứ. Metadata của lần chạy và đường dẫn file đầu ra được lưu tại bảng `ais.trajectory.hysplit_runs_bronze`, phục vụ xử lý tiếp theo, truy vết và chạy lại khi cần.

4.4.3 Trajectory processing

Job	Output	Chức năng
<code>hysplit_trajectory_parse_silver.py</code>	<code>ais.trajectory.hysplit_trajectories_silver</code>	Chuyển file trajectory thành các bản ghi gồm <code>run_id</code> , <code>age_h</code> , tọa độ, độ cao, áp suất và nhiệt độ.
<code>hysplit_trajectory_cluster_silver.py</code>	<code>ais.trajectory.hysplit_trajectories_clustered_silver</code>	Áp dụng k-means lên hình học đường đi để gom trajectory thành <i>k</i> cụm.
<code>openaq_spatial_gradient_silver.py</code>	<code>ais.features.openaq_spatial_gradient_silver</code>	Tính gradient PM2.5 theo các hướng Bắc, Nam, Đông và Tây từ mạng lưới trạm quanh Hà Nội.
<code>sentinel5p_grid_silver.py</code>	<code>ais.satellite.sentinel5p_grid_silver</code>	Chuyển sản phẩm Sentinel-5P hằng ngày thành lưới không gian để lấy mẫu dọc theo đường đi của khối khí.
<code>trajectory_path_sampling_silver.py</code>	<code>ais.features.trajectory_path_satellite_silver</code>	Lấy mẫu cột NO ₂ và chỉ số aerosol từ lưới Sentinel-5P dọc trajectory, tạo tín hiệu thành phần khí quyển tích hợp theo đường đi.
<code>trajectory_hourly_features_silver.py</code>	<code>ais.features.trajectory_hourly_features_silver</code>	Tổng hợp theo giờ các đặc trưng của cụm trajectory, vị trí điểm cuối, centroid vùng nguồn, trung bình NO ₂ /AER.

4.5 Machine learning

4.5.1 Mục tiêu ML

ML layer dự báo PM2.5 trong các horizon 6 giờ, 12 giờ và 24 giờ. Bài toán được mô hình hóa là hồi quy có giám sát:

Input: features tại thời điểm t
 Output: PM2.5 tại thời điểm $t + \text{horizon}$

4.5.2 Training dataset

`hanoi_pm25_training_dataset_gold.py` tạo training dataset từ gold master features:

1. Lọc bản ghi có đủ feature quan trọng.
2. Sinh nhãn `target_pm25_horizon_6h`, `target_pm25_horizon_12h`, `target_pm25_horizon_24h`.
3. Chia train/validation/test theo thời gian, có purge gap để tránh leakage.
4. Ghi dataset vào Iceberg features namespace kèm `dataset_version`, `feature_version`, `schema_hash`.

4.5.3 Model architecture

AIS sử dụng LightGBM. Gradient boosting được chọn vì phù hợp với dữ liệu tabular, xử lý missing value tốt, huấn luyện nhanh, dễ giải thích bằng feature importance và không yêu cầu lượng dữ liệu cực lớn như deep learning.

4.5.4 Training, promotion và prediction

Phase	Script	Input	Output	Key action
Training	<code>ml/train_hanoi_pm25.py</code>	Training dataset Gold	Model artifact + run metadata	Train candidate model, log metrics, feature importance
Promotion	<code>ml/promote_hanoi_pm25_model.py</code>	Model runs Gold	Model registry Gold	Chọn model đạt criteria, cập nhật production pointer

Phase	Script	Input	Output	Key action
Prediction	ml/prediction_hanoi_pm25.py	Cassandra Feature State + registry/artifact	Cassandra Latest Forecast	Load production model đúng <code>feature_version/schema_hash</code> , ghi forecast mới nhất

4.6 Storage, API và dashboard

4.6.1 Định dạng lưu trữ, nén và phân tầng dữ liệu nóng/lạnh

AIS phân tầng dữ liệu theo tần suất truy cập và kiểu truy vấn. Dữ liệu lịch sử, raw file lớn và các bảng Bronze/Silver/Gold được xem là **cold/historical data**, lưu trên HDFS và Iceberg để tối ưu cho batch processing, time travel, audit và huấn luyện mô hình. Dữ liệu mới nhất phục vụ API như latest PM2.5, forecast mới nhất và online feature state được xem là **hot data**, lưu trong Cassandra để truy vấn độ trễ thấp. Các sản phẩm visualization có payload lớn như heatmap, trajectory và source attribution map được materialize thành JSON/GeoJSON cache trên HDFS hoặc object storage.

Với dữ liệu dạng bảng, AIS ưu tiên định dạng cột như Parquet để tận dụng column pruning, predicate pushdown và nén dữ liệu. Các bảng Iceberg được partition theo ngày, nguồn, location hoặc horizon tùy query pattern. Compression giúp giảm dung lượng lưu trữ và chi phí đọc/ghi, trong khi compaction định kỳ giúp gom small files sinh ra từ streaming micro-batch.

Tính năng Iceberg	Ứng dụng trong AIS
ACID commit	Ghi table an toàn từ Spark batch/streaming
Snapshot/time travel	Debug, rollback, lineage
MERGE INTO	Idempotent upsert/deduplication
Partition pruning	Tối ưu đọc theo ngày, nguồn, horizon
Schema evolution	Thêm feature/field mới không phá pipeline
Metadata tables	Theo dõi file, snapshot, history
Compaction	Gom file nhỏ sau streaming

4.6.2 Storage serving design

Nhóm bảng serving:

Bảng	Query pattern	Partition key	Clustering key
pm25_latest_by_location	Lấy PM2.5/latest forecast mới nhất	location_id	event_timeDESC
pm25_timeseries_by_location	Time series ngắn theo location/time range	location_id, date_bucket	event_timeASC
pm25_forecast_by_location	Forecast theo location/horizon/date	location_id, forecast_date	horizon, target_time
station_observations_latest	Quan trắc mới nhất theo trạm	station_id	event_timeDESC
weather_hourly_by_location_day	Weather hourly theo ngày	location_id, date	hourASC
openaq_by_location_parameter_day	OpenAQ theo station/parameter/day	location_id, parameter, date	hourASC

Các sản phẩm nặng hơn được phục vụ bằng cache:

Sản phẩm	Storage serving	Lý do
PM2.5 heatmap GeoJSON	Cache JSON/GeoJSON	Payload lớn, ít cần query row-level
Backward trajectory paths	Cache GeoJSON	Dữ liệu đường/geometry, phù hợp file cache
Source attribution panel	Cache JSON + Cassandra latest metadata	Vừa cần map layer vừa cần summary nhanh
Visualization manifest	Cache JSON	API kiểm tra freshness/layer availability

4.6.3 API serving design

FastAPI services đọc từ Cassandra và cache đã materialize. API không tạo Spark session, không chạy ML inference trực tiếp tại request time và không gọi HYSPLIT online. Thiết kế này giúp request latency ổn định và tách serving khỏi compute workload.

AIS chuẩn hóa endpoint theo dạng `/api/v1/...`, dùng `/health` và `/ready`.

Endpoint	Method	Chức năng
<code>/health</code>	GET	Liveness check, kiểm tra process API
<code>/ready</code>	GET	Readiness check, kiểm tra Cassandra/cache/freshness

Endpoint	Method	Chức năng
/metrics	GET	Metrics phục vụ monitoring
/api/v1/hanoi/pm25/latest	GET	PM2.5 mới nhất và metadata freshness
/api/v1/hanoi/pm25/forecast/latest	GET	Forecast 6h/12h/24h mới nhất
/api/v1/hanoi/pm25/timeseries	GET	Time series quan trắc và forecast
/api/v1/visualization/manifest/latest	GET	Manifest cache và trạng thái layer
/api/v1/visualization/trajectories/backward/latest	GET	GeoJSON backward trajectories
/api/v1/visualization/source-attribution/latest	GET	Summary source attribution
/api/v1/visualization/stations/latest	GET	Station observation GeoJSON/latest

API được thiết kế theo query rõ ràng cho location/time/horizon, validate đầy đủ query parameter và trả response JSON kèm metadata như `generated_at`, `feature_version`, `model_version`, `freshness`. Các endpoint dùng cache header cho dữ liệu ít thay đổi, timeout/error handling thống nhất, và `/ready` trả lỗi khi cache quá stale hoặc dependency chính chưa sẵn sàng.

4.6.4 Dashboard UI

Dashboard React/Vite hiển thị realtime KPI cards cho PM2.5 hiện tại, forecast cards 6h/12h/24h, time series observed/forecast, PM2.5 heatmap layer, station observation markers, backward trajectory overlay, source attribution panel, data freshness indicator và trạng thái API/pipeline. UI kết nối API qua biến môi trường runtime, phù hợp cả Docker Compose và Kubernetes.

4.7 Airflow, Docker và Kubernetes

4.7.1 Vai trò Airflow

Airflow điều phối ingestion, batch, ML, serving và monitoring. Hệ thống giữ 8 DAG.

DAG	Vai trò
ais_pipeline_dag.py	Pipeline tổng thể/bootstrap/backfill
ais_era5_ingestion_dag.py	Ingest ERA5 surface/pressure định kỳ
ais_hanoi_silver_gold_dag.py	Tạo Silver/Gold cho Hà Nội

DAG	Vai trò
ais_pm25_k8s_compute_dag.py	Chạy compute/ML trên Kubernetes
ais_streaming_supervision_dag.py	Giám sát streaming jobs, lag, restart condition
ais_visualization_product_dag.py	Tạo visualization Gold và export cache
ais_trajectory_tier2_dag.py	ERA5→ARL→HYSPLIT→trajectory features
ais_maintenance_dag.py	Maintenance Iceberg, cleanup, compaction, reconciliation

Airflow quản lý dependency theo thứ tự từ kiểm tra infrastructure, đảm bảo Kafka topics, HDFS layout, Iceberg tables và Cassandra schema, sau đó ingest dữ liệu, giám sát streaming, chạy Silver/Gold jobs và refresh context khi có ERA5 pressure, HYSPLIT hoặc satellite mới.

Ở nhánh batch, DAG tạo training dataset, lưu model artifact và cập nhật production pointer; ở nhánh near-realtime, DAG giám sát audit path, Online Feature Builder, Prediction Job, Cassandra latest state, serving/API và metrics/alert.

DAG cấu hình retry theo từng task, timeout, SLA/freshness check, cảnh báo khi task fail, backfill theo khoảng thời gian và idempotent rerun theo partition. Các task ghi output đều có khóa nghiệp vụ và MERGE/UPSERT nên Airflow retry không tạo duplicate.

4.7.2 Docker Compose local development

`docker-compose.yml` cung cấp môi trường local full-stack để nhóm phát triển.

Service	Công nghệ	Vai trò
namenode	Hadoop HDFS	HDFS NameNode và WebHDFS UI
datanode	Hadoop HDFS	HDFS DataNode
kafka	Apache Kafka	Broker message queue
zookeeper	Kafka	Quản lý cluster metadata
spark-master	Apache Spark	Spark standalone master/UI cho local
spark-worker	Apache Spark	Spark worker/executor local
cassandra	Apache Cassandra	Serving database local
airflow-webserver	Apache Airflow	Airflow UI
airflow-scheduler	Apache Airflow	DAG scheduler
pm25-api	FastAPI/Uvicorn	PM2.5 API
visualization-api	FastAPI/Uvicorn	Visualization API

Service	Công nghệ	Vai trò
ais-ui	React/Vite/Nginx	Dashboard
monitoring	Streamlit/custom app	

4.7.3 Kubernetes deployment

Kubernetes manifests trong `deploy/k8s/` triển khai:

Thành phần	File/folder
Namespace	<code>deploy/k8s/namespace.yaml</code>
ConfigMap	<code>deploy/k8s/configmap.yaml</code>
Secret	<code>deploy/k8s/secret.example.yaml</code> và secret runtime
ServiceAccount/RBAC	<code>deploy/k8s/serviceaccount.yaml</code> , <code>rbac.yaml</code>
Spark submit	<code>deploy/k8s/spark/</code>
PM2.5 API	<code>deploy/k8s/api/</code>
UI	<code>deploy/k8s/ui/</code>
ML jobs	<code>deploy/k8s/ml/</code>
Checks	<code>deploy/k8s/checks/</code>
Kustomize	<code>deploy/k8s/kustomization.yaml</code>

Kubernetes được dùng để tách cấu hình vận hành khỏi code. Các service chạy trong namespace riêng, endpoint nội bộ được truy cập qua Kubernetes Service DNS, còn cấu hình môi trường như Kafka bootstrap servers, Iceberg warehouse, HDFS endpoint, Cassandra contact points và API base URL được inject qua ConfigMap. Secret tách riêng các giá trị nhạy cảm như database password, token và API key để container image không chứa credential.

Các workload được chia theo bản chất chạy: API/UI là Deployment có Service, readiness/liveness probe và rolling update; Spark/Silver/Gold/backfill/predict là Job hoặc task do Airflow submit; tác vụ định kỳ như maintenance, predict hoặc health check có thể chạy bằng CronJob. Resource request/limit được đặt theo profile của từng workload: API/UI cần CPU/RAM ổn định nhưng nhỏ, Spark driver cần đủ memory cho plan và metadata, executor cần memory/cores theo kích thước job, còn các check job dùng cấu hình nhẹ để không tranh tài nguyên với pipeline chính.

RBAC giới hạn quyền của service account theo namespace. Spark driver chỉ cần quyền tạo/xóa executor pod, đọc ConfigMap/Secret cần thiết và ghi log; API/UI không có quyền thao tác pod. Điều này giúp giảm rủi ro khi một container lỗi hoặc bị cấu hình sai.

4.7.4 Spark-on-Kubernetes

Spark jobs được submit lên Kubernetes. Cấu hình gồm:

- Driver pod.
- Executor pods.
- ServiceAccount/RBAC.
- ConfigMap environment.
- Resource request/limit.
- Image pull policy.
- Volume mount cho config.
- HDFS/Iceberg/Kafka/Cassandra connection config.
- Dynamic executor instances cho job lớn.

Khi chạy trên Kubernetes, mỗi Spark application có driver pod riêng và executor pods được tạo động theo cấu hình job. Driver nhận cấu hình qua ConfigMap/Secret, mount file cấu hình cần thiết, ghi log ra stdout để Kubernetes/Airflow thu thập, đồng thời kết nối tới Kafka, HDFS/Iceberg và Cassandra bằng service DNS hoặc endpoint được inject từ môi trường. Các tham số được đặt trong template submit để cùng một code có thể chạy ở local, staging hoặc cụm production-like.

Với các job lớn, nhóm ưu tiên submit theo cụm job liên quan thay vì tách quá nhỏ từng Spark application. Cách này giảm overhead tạo driver/executor nhiều lần, tận dụng lại Spark session/config/dependency và cho phép một số bước độc lập trong cùng application chạy song song khi không phụ thuộc dữ liệu lẫn nhau.

4.7.5 Chiến lược mở rộng tài nguyên

Như đã đề cập ở phía trên, do có giới hạn về tài nguyên phần cứng nên AIS chỉ được chạy và thực hiện trên cấu hình tối thiểu. Tuy nhiên, AIS hoàn toàn có thể mở rộng theo cả chiều ngang và chiều dọc. Với Spark, horizontal scaling được thực hiện bằng cách tăng số executor pod hoặc bật dynamic allocation cho các job lớn; vertical scaling được thực hiện bằng cách tăng CPU/RAM cho driver và executor theo profile của từng job. Với Kafka, hệ thống có thể tăng số broker và partition để tăng throughput và khả năng song song của consumer group. Với Cassandra, khả năng mở rộng đạt được bằng cách thêm node và thiết kế partition key phù hợp với query pattern. API/UI có thể scale ngang bằng cách tăng số replica trong Kubernetes Deployment.

5 Vận hành và kiểm thử

5.1 Monitoring

5.1.1 Monitoring scope

Nhóm metric	Ví dụ
Kafka	Lag theo topic/consumer group, throughput, error rate
Spark	Batch duration, input rows, processed rows/sec, state size, checkpoint health
Iceberg	Số file nhỏ, snapshot count, commit duration, table freshness
HDFS	Dung lượng, replication, path availability
Cassandra	Read/write latency, timeout, unavailable exception, table count
API	Request latency, status code, error rate, dependency health
ML	Prediction freshness, model version, feature drift, error after ground truth
Data quality	Null ratio, invalid count, duplicate count, late event count

5.1.2 Logging

Mỗi job ghi structured logs gồm `job_name`, `job_run_id`, `input_window`, `output_window`, `input_count`, `output_count`, `invalid_count`, `duplicate_count`, `duration`, `status`, `error_message`. Log giúp truy vết từ Airflow task đến Spark job và Iceberg snapshot.

5.1.3 Alerting

Alert được sinh khi Kafka lag vượt ngưỡng, Spark streaming batch duration vượt trigger interval, checkpoint lỗi hoặc không cập nhật, Iceberg table quá stale, API /ready trả lỗi, Cassandra timeout tăng, data quality vượt threshold hoặc model inference không có output mới.

5.2 Testing và data quality

5.2.1 Testing strategy

Test type	Scope	Implementation
Unit tests	Schema parsing, timezone normalization, unit conversion, QA bitmask logic	<code>pytest</code> cho function trong <code>ingest/</code> và <code>spark_jobs/</code>
Data quality tests	Range checks, coverage thresholds, freshness, null ratio, duplicate detection	Validation trong <code>Silver/Gold jobs</code> và <code>quality_checks.py</code>
Spark transform tests	Input DataFrame nhỏ → output kỳ vọng	Local Spark session hoặc test container
Streaming integration tests	Kafka publish → Spark streaming → Iceberg Bronze	Docker Compose test environment
Incremental parity tests	Full refresh output == incremental output	So sánh snapshot/output theo date window
Point-in-time tests	Không feature nào có <code>available_at > prediction_origin</code>	Assertion trong training/serving feature job
Model regression tests	Candidate model vượt baseline và không tệ hơn threshold	Promotion step kiểm tra metric
API tests	Health, ready, schema, freshness, error cases	<code>pytest</code> + HTTP client
Deployment	Docker Compose up, topic tồn tại, HDFS layout, Iceberg namespace	<code>scripts/check_*.sh</code> , <code>scripts/check_*.py</code>

5.2.2 Data quality rules

Data quality checks được chia theo nguồn:

- OpenAQ: PM2.5 range, unit, station_id, duplicate, coverage.
- Weather: null rate, wind range, temperature range, precipitation non-negative.
- ERA5: file availability, variable list, checksum, spatial bounds.
- Sentinel-5P: QA flag, valid pixel percentage, cloud/coverage.
- MAIAC: QA bitmask, AOD range, tile coverage.
- Feature Gold: missing feature ratio, schema hash, target availability, leakage check.
- Forecast: horizon completeness, model version, prediction range, freshness.

5.3 Security và governance

5.3.1 Secret management

Credential, API key, database password và token không được hard-code trong source code. Hệ thống dùng `.env.example` cho local template, Docker Compose env file cho dev, Kubernetes Secret cho runtime và biến môi trường để inject config vào container.

5.3.2 Access control

Thành phần	Cơ chế
API	Token/API key hoặc gateway auth nếu public
Cassandra	Username/password, role theo keyspace
Kafka	Có thể bật SASL/SSL trong production
HDFS/Iceberg	Phân quyền theo user/service account trong cluster
Kubernetes	RBAC theo namespace/service account
Airflow	User/role, connection secret, DAG permission

5.3.3 Data governance

AIS ghi metadata phục vụ lineage và audit như `dataset_version`, `feature_version`, `schema_hash`, `model_version`, `job_run_id`, `source`, `event_id`, `input_snapshot_id` và `output_snapshot_id`. Nhờ đó một forecast có thể truy vết về model, feature set, dataset version và snapshot dữ liệu đầu vào.

5.4 Tối ưu hiệu năng

5.4.1 Spark config

AIS bật Adaptive Query Execution, điều chỉnh shuffle partition theo workload, dùng broadcast join cho dimension nhỏ và tận dụng Parquet vectorized reader, predicate pushdown, column pruning, dynamic partition pruning. Executor memory/cores được đặt theo từng job profile; streaming job dùng `maxOffsetsPerTrigger` để giới hạn tốc độ đọc Kafka.

5.4.2 Tối ưu dữ liệu và join

AIS partition bảng Iceberg theo ngày/nguồn/horizon, pre-aggregate PM2.5 theo giờ trước khi join, broadcast station/grid metadata và repartition theo key lớn khi cần. Feature lag/lead dùng window functions thay vì self-join; khi có skew có thể dùng salting key, còn small files sau streaming được xử lý bằng Iceberg compaction định kỳ.

5.4.3 Tối ưu streaming

Streaming job dùng watermark để giới hạn state size, deduplicate theo key phù hợp, checkpoint riêng cho từng job và DLQ cho record lỗi. Hệ thống theo dõi batch duration so với trigger interval; Iceberg compaction được chạy off-peak để giảm small files mà không ảnh hưởng luồng streaming chính.

5.4.4 Cache và persistence trong Spark

Cache/persistence được dùng có chọn lọc, không cache toàn bộ pipeline. Nguyên tắc của AIS là chỉ persist DataFrame khi cùng một kết quả trung gian được tái sử dụng nhiều lần trong cùng job và chi phí tính lại lớn hơn chi phí giữ trong bộ nhớ/đĩa.

DataFrame có thể cache/persist	Khi nào dùng	Khi nào không dùng
OpenAQ sau cleaning	Được dùng nhiều lần để tính lag, rolling statistics, coverage và spatial gradient	Không cache nếu chỉ ghi một lần sang Silver
Master feature intermediate	Được reuse cho training dataset, serving feature và visualization aggregate trong cùng batch run	Không cache nếu job tách riêng và đọc lại từ Iceberg đã partition tốt
Station/grid metadata	Có thể broadcast/cache vì nhỏ và được join lặp lại	Không cần persist nếu Spark tự broadcast hiệu quả
Trajectory waypoint sau parse	Dùng khi vừa cluster, vừa sample satellite, vừa aggregate residence signal	Không cache nếu chỉ chạy một bước parse → write

Với dữ liệu vừa phải, AIS ưu tiên **MEMORY_AND_DISK** thay vì chỉ **MEMORY_ONLY** để tránh executor bị thiếu RAM. Sau khi hoàn thành các action cần thiết, job gọi `unpersist()` để giải phóng bộ nhớ, tránh ảnh hưởng các stage sau.

6 Kết quả triển khai

6.1 Kết quả chức năng

Hệ thống đã hoàn thiện luồng xử lý chính từ ingestion đến dashboard:

Mục tiêu	Kết quả
Multi-source ingestion	Hoàn thành adapters cho OpenAQ, WeatherAPI, ERA5, Sentinel-5P, MAIAC
Kafka-based ingestion	Hoàn thành topics và producer/consumer contract
Spark Structured Streaming	Hoàn thành 5 streaming jobs với checkpoint, watermark, dedup, MERGE
Bronze/Silver/Gold lakehouse	Hoàn thành phân tầng Iceberg trên HDFS
Hanoi feature engineering	Hoàn thành master feature table và serving feature table
Trajectory/source attribution	Hoàn thành pipeline ERA5→ARL→HYSPLIT→trajectory features
ML forecast	Hoàn thành train/promote/predict cho 6h/12h/24h
Serving/API	Hoàn thành API versioned, Cassandra serving và cache visualization
Dashboard	Hoàn thành realtime, historical và map dashboard
Orchestration	Hoàn thành 8 Airflow DAG
Deployment	Hoàn thành Docker Compose và Kubernetes manifests production-like
Observability/testing/security	Hoàn thành các cơ chế cốt lõi trong phạm vi project

6.2 Kết quả thực nghiệm ở mức project

Trong môi trường triển khai rút gọn, hệ thống chứng minh được các năng lực quan trọng:

- Ingestion adapters có thể publish event theo schema contract vào Kafka.
- Spark streaming đọc Kafka, xử lý late/duplicate event và ghi Bronze idempotent.
- Silver/Gold jobs tạo được bảng feature phục vụ ML và visualization.
- Model lifecycle tách training, promotion và inference.
- API đọc dữ liệu đã materialize thay vì chạy Spark/ML tại request time.
- Dashboard hiển thị được PM2.5, forecast, heatmap, trajectory và freshness.
- Airflow điều phối dependency và hỗ trợ retry/backfill.

6.3 Điểm mạnh

- Kiến trúc lakehouse rõ ràng, tách source of truth khỏi serving layer.
- Kiến trúc Lambda được hiệu chỉnh với historical batch và hai near-realtime path chạy đồng thời, phù hợp với dữ liệu khí quyển nhiều tốc độ.
- Feature engineering dựa trên nền tảng khí tượng, không chỉ thống kê đơn giản.
- Point-in-time correctness giúp tránh leakage trong ML.
- Serving materialized giúp API nhanh và ổn định.
- Idempotency cho phép rerun/backfill an toàn.
- HYSPLIT giúp tăng khả năng giải thích hướng vận chuyển ô nhiễm.

6.4 Điểm cần cải thiện chính

- Cấu hình triển khai hiện tại dùng 1 Kafka broker và 1 Cassandra node để phù hợp phần cứng.
- Một số kiểm thử tải lớn và chaos testing cần cluster/cloud thật.
- ERA5/Sentinel/MAIAC có độ trễ nguồn tự nhiên, nên near-real-time inference phải dùng proxy feature.
- Chưa kiểm thử đầy đủ kịch bản mất node hoặc mất volume; production cần back-up/restore plan cho Cassandra, HDFS/Iceberg warehouse, model artifact và Airflow metadata.

7 Bài học kinh nghiệm

7.0.1 Bài học 1: Dữ liệu đa nguồn cần quản lý độ trễ, chất lượng và phiên bản

Vấn đề Dữ liệu khí quyển từ nhiều nguồn không đến cùng thời điểm. Một số nguồn gần thời gian thực như OpenAQ và WeatherAPI có thể ingest nhanh, trong khi ERA5, Sentinel-5P, MAIAC hoặc kết quả trajectory thường có độ trễ lớn hơn. Nếu đưa tất cả vào một luồng xử lý duy nhất, pipeline dễ bị chờ dữ liệu, sai freshness hoặc làm chậm serving.

Giải pháp Nhóm tách luồng dữ liệu theo độ trễ và mục đích sử dụng: dữ liệu realtime/gần realtime đi qua Kafka và Spark Streaming để cập nhật Bronze nhanh; dữ liệu chậm hơn được xử lý bằng batch/backfill và cập nhật Silver/Gold khi đủ điều kiện. Các trường thời gian như `event_time`, `ingest_time`, `available_at` được dùng để đảm bảo point-in-time correctness.

Điểm rút ra Với Big Data pipeline, không nên ép mọi nguồn vào cùng một tốc độ xử lý. Dữ liệu đến muộn cần có luồng riêng, cơ chế backfill và logic cập nhật rõ ràng.

7.0.2 Bài học 2: Lỗi runtime cần data lineage để truy vết

Vấn đề Khi hệ thống chạy nhiều job liên tiếp, lỗi runtime có thể xuất hiện ở ingestion, Spark, Iceberg, Cassandra hoặc API. Nếu chỉ nhìn log lỗi tại job cuối, nhóm khó biết dữ liệu đầu vào đến từ nguồn nào, được xử lý bởi job nào, dùng schema/version nào và lỗi bắt đầu từ bước nào.

Giải pháp Nhóm bổ sung data lineage vào pipeline thông qua các metadata như `source`, `event_id`, `job_run_id`, `dataset_version`, `schema_hash`, `input_snapshot_id` và `output_snapshot_id`. Mỗi job log input/output count, thời gian xử lý và trạng thái ghi dữ liệu để có thể truy vết từ API/Gold ngược về Silver, Bronze và nguồn dữ liệu ban đầu.

Điểm rút ra Trong hệ thống dữ liệu lớn, lineage không chỉ phục vụ audit mà còn là công cụ debug chính khi lỗi runtime xảy ra liên tục.

7.0.3 Bài học 3: Deploy trên Docker Compose trước K8s là một con dao hai lưỡi

Vấn đề Docker Compose giúp dựng nhanh toàn bộ hệ thống trên máy cá nhân, thuận tiện để kiểm tra luồng dữ liệu end-to-end và phân biệt lỗi code với lỗi tích hợp. Tuy nhiên, sự đơn giản này cũng dễ tạo cảm giác rằng hệ thống đã sẵn sàng để triển khai. Khi chuyển sang Kubernetes, nhóm phải xử lý thêm service discovery, ConfigMap/Secret, resource

request/limit, volume, readiness/liveness probe, network policy và vòng đời riêng của từng workload. Một cấu hình chạy ổn định trên Docker Compose vì vậy chưa chắc hoạt động đúng hoặc mở rộng tốt trên Kubernetes.

Giải pháp Nhóm vẫn sử dụng Docker Compose để phát triển nhanh, xác nhận dependency và kiểm thử tích hợp ban đầu, nhưng đồng thời thiết kế ứng dụng theo hướng không phụ thuộc môi trường local. Cấu hình được tách khỏi code, dữ liệu bền vững không phụ thuộc container, các service có health check rõ ràng và tài nguyên được khai báo riêng.

Điểm rút ra Docker Compose là con dao hai lưỡi: nó rút ngắn đáng kể thời gian phát triển và debug, nhưng có thể che giấu các vấn đề chỉ xuất hiện trong môi trường phân tán. Vì vậy, Docker Compose nên được dùng để kiểm chứng chức năng, còn các yêu cầu về triển khai, khả năng phục hồi và mở rộng cần được kiểm tra trên Kubernetes từ sớm.

7.0.4 Bài học 4: Storage layout quyết định lớn đến hiệu năng và chi phí

Vấn đề Khi dữ liệu trong lake tăng lên, truy vấn trực tiếp trên Iceberg có thể chậm nếu bảng tổ chức chưa tốt: partition chưa phù hợp, nhiều small files, đọc thừa cột, hoặc truy vấn không tận dụng được pruning. Điều này ảnh hưởng đến batch job downstream và các bước export dữ liệu serving.

Giải pháp Nhóm tối ưu cách tổ chức Iceberg table theo đặc điểm truy vấn: partition theo ngày/nguồn/horizon, dùng schema rõ ràng cho Bronze/Silver/Gold, hạn chế đọc thừa cột, và chạy maintenance/compaction để giảm small files. Các bảng phục vụ API được materialize sang Cassandra/cache thay vì truy vấn lake trực tiếp tại request time.

Điểm rút ra Lakehouse không chỉ là nơi lưu dữ liệu. Hiệu năng phụ thuộc lớn vào cách thiết kế partition, file layout, schema và chiến lược materialize dữ liệu.

7.0.5 Bài học 5: Spark job cần cân bằng giữa modularity, hiệu năng và tài nguyên

Vấn đề Ban đầu mỗi Spark job được triển khai như một Spark application riêng. Cách này dễ quản lý từng job nhưng làm tăng overhead khởi tạo Spark session, submit job, load dependency và đọc lại dữ liệu trung gian. Khi nhiều job nhỏ chạy nối tiếp, tổng thời gian pipeline bị kéo dài.

Giải pháp Nhóm gom các Spark job có cùng ngữ cảnh xử lý vào cùng một Spark application để chạy theo cụm và tận dụng lại Spark session, config, dependency và dữ liệu

trung gian. Một số bước độc lập trong cùng app có thể chạy song song khi không phụ thuộc dữ liệu lẫn nhau, giúp giảm overhead và cải thiện thời gian pipeline.

Điểm rút ra Thiết kế Spark application cần cân bằng giữa modularity và overhead vận hành. Với nhiều job nhỏ liên quan chặt chẽ, gom cụm và chạy song song hợp lý sẽ hiệu quả hơn tách thành quá nhiều app riêng.

8 Phân công công việc

Thành viên	MSSV	Vai trò	Chi tiết công việc
Lê Minh Hoàng	20230028	Trưởng nhóm	Điều phối nhóm, phân chia task và theo dõi tiến độ
			Xây dựng Airflow DAG, dependency, retry/backfill; merge code, debug và kiểm tra end-to-end Triển khai Spark-on-Kubernetes, PM2.5 API, hỗ trợ visualization và demo
Nguyễn Ngọc Ánh	20235013	Thành viên	Join Silver để tạo Gold layer cho training/serving
			Xây dựng master features, training dataset và online features cho PM2.5. Chạy HYSPLIT, parse output và clustering trajectory Xây dựng Kubernetes base manifests; hỗ trợ báo cáo và slide
Nguyễn Minh Tùng	20235239	Thành viên	Tạo cấu hình pipeline, Iceberg schema và bootstrap table
			Xây dựng pipeline ERA5 từ ingestion đến Bronze/Silver Triển khai Spark-on-Kubernetes runtime, serving feature job, model registry và API deployment Hỗ trợ visualization/dashboard, báo cáo và slide
Nguyễn Ngọc Ánh	20235012	Thành viên	Xây dựng Silver layer cho WeatherAPI và MAIAC
			Tính OpenAQ spatial gradient, Sentinel-5P grid; migration Spark jobs sang Kubernetes Cập nhật schema/table serving, xây dựng station observations, backward trajectory và forward plume
Phạm Hoàng Tiến	20230071	Thành viên	Xây dựng Silver layer cho OpenAQ và Sentinel-5P
			Tạo Iceberg tables cho trajectory/HYSPLIT; ingest ERA5 pressure-level sang ARL Thiết kế target data flow, Config/Iceberg schema và forecast dashboard

9 Kết luận

Dự án **Atmospheric Intelligence System – AIS** đã xây dựng được một nền tảng dữ liệu lớn hoàn chỉnh và có khả năng vận hành cho bài toán giám sát, dự báo PM2.5 tại Hà Nội. Kiến trúc giải quyết đầy đủ năm đặc trưng 5V: lưu trữ lịch sử trên Iceberg và HDFS có khả năng mở rộng tới quy mô petabyte; tiếp nhận dữ liệu liên tục qua Kafka và Spark Structured Streaming; xử lý sáu định dạng gồm JSON, CSV, GeoJSON, NetCDF4, HDF5 và NOAA ARL; kiểm soát chất lượng qua nhiều tầng; đồng thời tạo ra giá trị thực tế thông qua giám sát gần thời gian thực, dự báo tại nhiều mốc, phân tích trajectory và dashboard sẵn sàng phục vụ vận hành.

Sự kết hợp giữa Apache Kafka, Spark, Iceberg, Cassandra, Airflow, HYSPLIT và Kubernetes tạo nên một kiến trúc có thể mở rộng theo chiều ngang và bảo trì độc lập ở từng lớp. Việc tách rõ ingestion, lưu trữ lịch sử, feature engineering, huấn luyện mô hình, suy diễn và serving giúp giảm phụ thuộc chặt giữa các thành phần. Nhờ đó, vấn đề chất lượng dữ liệu từ nguồn hoặc sự cạnh tranh tài nguyên tính toán ít có khả năng làm gián đoạn toàn bộ hệ thống. Cấu hình demo hiện tại được rút gọn để phù hợp với phần cứng, nhưng thiết kế có thể mở rộng sang cụm nhiều nút khi có đủ tài nguyên.

Không chỉ minh họa cách xây dựng một hệ thống Big Data hiện đại, AIS còn hướng đến một vấn đề sức khỏe cộng đồng thực tế. Các dự báo PM2.5 dễ tiếp cận, có độ chính xác được đánh giá rõ ràng, cùng phân tích hành lang vận chuyển tiềm năng có thể hỗ trợ người dân, cơ quan y tế và đơn vị quy hoạch đô thị đưa ra quyết định phù hợp trong các đợt ô nhiễm không khí. Kiến trúc của hệ thống cũng là nền tảng có thể tái sử dụng để mở rộng sang các đô thị khác tại Việt Nam, theo dõi thêm các chất ô nhiễm và phát triển những ứng dụng cảnh báo sớm.

10 Tài liệu tham khảo

- [1] Apache Software Foundation. *Apache Kafka Documentation*. <https://kafka.apache.org/documentation/>, 2024.
- [2] Apache Software Foundation. *Apache Spark Documentation*. <https://spark.apache.org/docs/latest/>, 2024.
- [3] Apache Iceberg Project. *Apache Iceberg Documentation*. <https://iceberg.apache.org/docs/latest/>, 2024.
- [4] Apache Software Foundation. *Apache Airflow Documentation*. <https://airflow.apache.org/docs/>, 2024.
- [5] OpenAQ Platform. *OpenAQ API v3 Documentation*. <https://docs.openaq.org/>, 2024.
- [6] H. Hersbach et al. “The ERA5 global reanalysis.” *Quarterly Journal of the Royal Meteorological Society*, 146(730):1999–2049, 2020.
- [7] NOAA Air Resources Laboratory. *HYSPLIT User’s Guide*. <https://www.ready.noaa.gov/hysplit.php>, 2024.
- [8] European Space Agency. *Sentinel-5P Mission and TROPOMI Products*. <https://sentinel.esa.int/web/sentinel/missions/sentinel-5p>, 2024.
- [9] NASA LP DAAC. *MCD19A2 MODIS/Terra+Aqua Land Aerosol Optical Depth Daily L2G Global 1km SIN Grid*. <https://doi.org/10.5067/MODIS/MCD19A2.006>, 2024.
- [10] G. Ke et al. “LightGBM: A Highly Efficient Gradient Boosting Decision Tree.” *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017.
- [11] World Health Organization. *WHO Global Air Quality Guidelines: Particulate Matter (PM_{2.5} and PM₁₀), Ozone, Nitrogen Dioxide, Sulfur Dioxide and Carbon Monoxide*. WHO, Geneva, 2021.